

CHAPTER 12

12. NestJS + Prisma — 스키마 파일에서 타입까지

Node.js · Express · NestJS 강의 · 2026

11장에서는 같은 게시판 도메인(User·Post의 1:N)을 TypeORM으로 만들었다. 엔티티 클래스에 `@Entity · @Column` 을 붙이면 그 클래스가 곧 테이블 설계도였고, 그 클래스를 그대로 타입으로 썼다. 이 장은 **똑같은 도메인, 똑같은 엔드포인트를 ORM만 Prisma로 바꿔 다시 만든다.** 바뀌는 건 데이터 계층 한 겹뿐이다. 컨트롤러와 DTO는 글자까지 그대로다. 그래서 두 장을 나란히 놓고 보면 ORM을 바꾸는 일이 코드에서 어디까지를 건드리는지 또렷이 보인다.

Prisma의 출발점은 클래스가 아니라 **schema.prisma** 라는 파일 하나다. 모델도, 관계도, DB 접속 정보도 전부 이 파일에 적는다. 그리고 이 파일을 기준으로 두 가지를 **생성**한다 — DB 테이블을 만드는 마이그레이션 SQL과, 코드에서 쓸 타입 안전 클라이언트다. "스키마 파일이 진실의 원천이고, 나머지는 거기서 만들어 낸다." 이 한 문장이 이 장 전체를 관통한다.

이 자료는 **개념 연습용**이다. 실제 수업에서 쓰는 파일 이름과 폴더 구성, 다루는 범위는 강사에 따라 다를 수 있다. 그래서 여기서는 파일이 아니라 **무엇을 배우는가**에 초점을 둔다.

이 장에서 배우는 것

- `schema.prisma` 한 파일로 모델·관계·datasource를 정의하는 법
- `prisma migrate dev` (테이블 생성)와 `prisma generate` (타입 생성)의 2단계 워크플로우
- `PrismaClient` 를 상속해 NestJS 생명주기에 연결하는 방법
- `@Global` 모듈로 DB 접근을 한 번만 등록하고 전역에서 주입받는 구조
- `this.prisma.user.findMany({ include: { posts: true } })` — 타입 자동완성이 붙는 쿼리

코드는 11장과 같은 구조를 따른다. 컨트롤러·DTO·모듈 골격은 그대로고, 달라진 건 스키마 정의의 자리와 서비스의 DB 접근 방식이다. 포트는 3002 다(11장 TypeORM 앱과 동시에 띄워 비교하라고 일부러 다르게 뒀다).

주제	핵심
스키마 정의	<code>schema.prisma</code> 한 파일에 모델·관계·datasource
마이그레이션	<code>migrate dev</code> 가 만드는 SQL 변경 이력
클라이언트 연결	<code>PrismaClient</code> 상속 + 생명주기 연결
전역 등록	<code>@Global</code> — 한 번 등록으로 전역 주입
쿼리	<code>prisma.user.*</code> — 타입 자동완성, <code>include</code> 로 관계 로드

핵심 개념 — 스키마 파일이 진실의 원천이다

TypeORM과 Prisma는 "스키마를 어디에 적느냐"부터 갈린다. TypeORM은 엔티티 클래스가 곧 스키마였다. 클래스 안의 `@Column` 데코레이터가 컬럼이고, 클래스 자체가 타입이다. 코드와 스키마가 한 몸이다.

Prisma는 둘을 분리한다. 스키마는 `schema.prisma` 라는 **별도 파일**에 Prisma 전용 문법으로 적는다. 자바스크립트도 타입스크립트도 아니다. 이 파일이 진실의 원천이고, 코드에서 쓸 타입은 여기서 **생성(generate)** 한 결과물이다.

이 "생성" 단계가 Prisma의 정체성이다. 흐름은 이렇다.

```

schema.prisma  —prisma migrate dev—▶  DB에 테이블 생성 (+ 마이그레이션 SQL 기록)
      |
      └── —prisma generate —▶  @prisma/client (타입 완비된 클라이언트)
  
```

`schema.prisma` 를 고치면 이 두 명령을 다시 돌려 DB와 타입을 맞춘다. 스키마를 한 곳에 모아 두고, DB와 코드 양쪽을 거기서 동기화하는 방식이다.

두 ORM의 차이를 의식하며 읽자 — 이 장의 코드는 11장 TypeORM 코드와 한 줄씩 견뎌 보면 가장 빨리 들어온다. "여기서 타입이 자동 생성된다", "여기에 DB 관련 import가 없다" 같은 지점이 두 ORM이 갈라지는 자리다. 어디가 같고 어디가 다른지를 짚으며 읽으면 ORM 교체가 코드의 어느 층까지 닿는지가 보인다.

준비

먼저 로컬 PostgreSQL에 이 장 전용 빈 DB를 하나 만든다.

```
createdb nest_prisma
```

그다음 의존성을 설치하고 환경변수 파일을 만든다(예제 폴더의 `.env.example` 을 복사해 `.env` 로 둔다).

```
npm install
cp .env.example .env
```

Prisma는 접속 정보를 `schema.prisma` 의 `datasource` 가 아니라 `.env` 의 `DATABASE_URL` 에서 읽는다. 복사한 `.env` 를 본인 계정으로 고친다.

```
DATABASE_URL="postgresql://USER:PASSWORD@localhost:5432/nest_prisma?schema=public"
```

`USER:PASSWORD` 자리에 본인 로컬 PostgreSQL 계정을 넣는다. 비밀번호가 없는 계정이면

`postgresql://USER@localhost:5432/nest_prisma?schema=public` 처럼 `:PASSWORD` 를 통째로 뺀다. 11장이 `DB_HOST · DB_USER` 같은 변수를 따로따로 받았던 것과 달리, Prisma는 접속 정보 전체를 이 URL 한 줄로 받는다.

이제 핵심인 **2단계 워크플로우**다. 스키마 파일을 DB와 타입에 반영하는 두 명령이다.

```
# ① 마이그레이션 생성 + DB 적용 - schema.prisma를 보고 테이블을 만든다
npm run prisma:migrate      # = prisma migrate dev--name init

# ② 타입이 완비된 Prisma Client 생성 - @prisma/client에 타입을 채운다
npm run prisma:generate     # = prisma generate
```

①의 `migrate dev` 는 끝에 `generate` 를 자동으로 한 번 돌린다. 그래서 처음 한 번은 ①만으로도 타입이 생긴다. 다만 스키마를 그대로 두고 타입만 다시 만들고 싶을 때(예: 클론 직후)는 ②를 단독으로 쓴다. 데이터를 눈으로 확인하고 싶으면 내장 GUI를 띄운다.

```
npm run prisma:studio      # = prisma studio - localhost:5555
```

마지막으로 서버를 실행한다. 포트는 `3002` 다.

```
npm run start:dev          # → [Prisma] http://localhost:3002
```

⚠ **generate** 를 한 번도 안 돌리면 빌드가 깨진다. `@prisma/client` 는 빈 껍데기로 설치되고, 타입은 `generate` 가 채운다. 그 전에 `npm run start:dev` 부터 하면 `User · Post` 타입을 못 찾아 컴파일이 실패한다. 설치 후엔 마이그레이션 먼저, 그다음 실행이 순서다.

스키마 정의 — 단일 진실 공급원

모든 게 시작되는 파일이 `schema.prisma` 다. 위쪽 절반은 "이 스키마로 무엇을 만들지", 아래 절반은 "데이터가 어떻게 생겼는지"를 적는다.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

- `generator client` — `prisma generate` 가 무엇을 만들지 정한다. `prisma-client-js` 는 "자바스크립트/타입스크립트용 클라이언트를 만들라"는 뜻이다. 우리가 코드에서 `import { PrismaClient } from "@prisma/client"` 로 가져다 쓰는 그 클라이언트가 여기서 나온다.
- `datasource db` — 어떤 DB에 붙을지 정한다. `provider` 는 `postgresql` , `url` 은 `env("DATABASE_URL")` — 즉 `.env` 에서 읽는다. 접속 문자열을 코드나 스키마에 직접 박지 않고 환경변수로 빼는 자리다.

이제 데이터 모양이다. TypeORM의 엔티티 클래스에 해당하는 부분이 `model` 이다.

```
model User {
  id      Int      @id @default(autoincrement())
  email   String   @unique
  name    String
  posts   Post[]   // 1:N - 관계를 양쪽 모델에 선언한다
  createdAt DateTime @default(now())
}
```

한 줄이 한 컬럼이다. 필드명 타입 @속성 순서로 읽으면 된다.

- `id Int @id @default(autoincrement())` — 정수형 기본키. `@default(autoincrement())` 로 새 행마다 1 씩 자동 증가한다. TypeORM의 `@PrimaryGeneratedColumn` 과 같은 역할이다.
- `email String @unique` — 문자열이고 중복을 허용하지 않는다. 같은 이메일로 두 번 가입하려 들면 DB가 막는다.
- `name String` — 이름. 속성이 없으니 일반 컬럼이다.
- `posts Post[]` — 관계 필드다. 한 사용자가 여러 글을 가지므로 `Post[]` (배열)다. 이 줄은 실제 컬럼이 아니라 "User에서 Post 쪽으로 이어지는 1:N 관계가 있다"는 선언이다. 코드에서 `include: { posts: true }` 로 글 목록을 함께 꺼낼 때 이 필드를 쓴다.
- `createdAt DateTime @default(now())` — 생성 시각. `@default(now())` 로 행이 만들어질 때의 시각이 자동으로 채워진다.

```
model Post {
  id      Int      @id @default(autoincrement())
  title   String
  content String
  // N:1 - authorId(FK)로 User와 연결. 작성자 삭제 시 함께 삭제(Cascade).
  author  User     @relation(fields: [authorId], references: [id], onDelete: Cascade)
  authorId Int

  createdAt DateTime @default(now())
}
```

`Post` 는 반대편, 즉 N:1 쪽이다. 관계의 핵심은 두 줄이다.

- `author User @relation(. )` — 이 글의 작성자(`User`)로 이어지는 관계 필드다. `@relation` 의 `fields: [authorId]` 는 "이 모델의 `authorId` 컬럼을", `references: [id]` 는 "User의 `id` 에 맞춘다"는 뜻이다. 데이터베이스의 외래키(FK) 연결을 이렇게 적는다.
- `authorId Int` — 실제로 DB 테이블에 생기는 외래키 컬럼이다. `author` 는 코드에서 객체로 접근하는 통로, `authorId` 는 DB에 저장되는 숫자값이다. 그래서 글을 만들 때는 `authorId` (숫자)만 넘기면 된다.
- `onDelete: Cascade` — 작성자가 삭제되면 그 사람의 글도 함께 지운다. 이게 없으면 글이 남은 사용자는 삭제가 막힌다.

관계는 **양쪽 모델에 모두 선언**한다는 점을 짚어 두자. `User` 의 `posts Post[]` 와 `Post` 의 `author User` 가 한 쌍이다. 한쪽만 적으면 Prisma가 관계를 완성하지 못한다.

짚고 가기 — 마이그레이션 폴더는 처음엔 저장소에 없다. `npm run prisma:migrate` 를 돌리는 순간 Prisma가 위 스키마를 보고 `CREATE TABLE..` SQL 파일을 그 안에 만들어 넣는다. 이 SQL이 곧 "DB를 어떻게 바꿨는가"의 기록이다. 스키마를 고칠 때마다 새 마이그레이션이 쌓여 변경 이력이 된다.

클라이언트를 생명주기에 연결

`schema.prisma` 에서 `generate` 로 만들어진 `PrismaClient` 는 DB와 대화하는 객체다. 그런데 이 객체는 한 가지를 직접 챙겨야 한다. **언제 DB에 연결하고 언제 끊을지**다. NestJS의 생명주기에 그 두 시점을 맞추는 서비스를 둔다.

```
import { Injectable, OnModuleInit, OnModuleDestroy } from "@nestjs/common";
import { PrismaClient } from "@prisma/client";

@Injectable()
export class PrismaService
  extends PrismaClient
  implements OnModuleInit, OnModuleDestroy
{
  async onModuleInit() {
    await this.$connect();
  }

  async onModuleDestroy() {
    await this.$disconnect();
  }
}
```

- `extends PrismaClient` — 이 서비스가 `PrismaClient` 를 상속한다. 그래서 인스턴스는 `prisma.user`, `prisma.post`, `prisma.$connect` 같은 클라이언트의 모든 기능을 그대로 가진다. 별도의 래핑 없이 클라이언트 그 자체에 NestJS 생명주기 혹은 더한 형태다.
- `@Injectable()` — 이 클래스를 NestJS가 관리하는 주입 대상으로 만든다. 서비스들이 생성자에서 이걸 받아 쓸 수 있게 된다.
- `implements OnModuleInit, OnModuleDestroy` — NestJS가 제공하는 두 생명주기 훅이다. 모듈이 초기화될 때 `onModuleInit`, 앱이 내려갈 때 `onModuleDestroy` 가 호출된다.
- `onModuleInit` 에서 `this.$connect()` — 앱이 뜰 때 DB에 연결한다.
- `onModuleDestroy` 에서 `this.$disconnect()` — 앱이 종료될 때 연결을 정리한다. 끊지 않고 죽으면 DB 쪽에 연결이 남아 새다.

11장에서는 이 "연결" 책임이 루트 모듈의 `TypeOrmModule.forRoot({})` 에 들어 있었다. `host:port:username`을 거기 적고 NestJS가 알아서 연결했다. Prisma는 접속 정보를 `.env + schema.prisma` 로 빼고, 연결을 켜고 끄는 일만 이 작은 서비스가 맡는다.

@Global 로 한 번만 등록

위에서 만든 연결 서비스를 모든 모듈이 쓸 수 있게 푸는 모듈이다.

```
import { Global, Module } from "@nestjs/common";
import { PrismaService } from "../prisma.service";

@Global()
@Module({
  providers: [PrismaService],
  exports: [PrismaService],
})
export class PrismaModule {}
```

- `providers: [PrismaService]` — 이 모듈이 `PrismaService` 를 만들어 제공한다.
- `exports: [PrismaService]` — 만든 걸 다른 모듈도 가져다 쓰도록 내보낸다. `exports` 가 없으면 이 모듈 안에 갇힌다.
- `@Global()` — 핵심이다. 이걸 붙이면 앱 어디서든 `PrismaService` 를 주입받을 수 있다. 사용자 모듈이든 게시글 모듈이든 따로 import할 필요가 없다.

11장 TypeORM과 가장 크게 갈리는 지점이 여기서이다. TypeORM은 모듈마다 `TypeOrmModule.forFeature([User])` 를 적어, 그 모듈이 쓸 엔티티의 리포지토리를 일일이 등록해야 했다. Prisma는 `@Global` 모듈 하나로 끝낸다. 그래서 자원 별 모듈에는 DB 관련 import가 한 줄도 없다.

```
// 사용자 모듈 - TypeOrmModule.forFeature가 없다!
@Module({
  controllers: [UsersController],
  providers: [UsersService],
})
export class UsersModule {}
```

`forFeature` 가 사라졌다. `PrismaService` 는 전역 모듈에서 오므로 모듈 정의가 간결해진다.

쿼리 — `this.prisma.user.*`

비즈니스 로직이 실제로 DB를 건드리는 곳이 서비스다. 이 계층이 11장과 가장 많이 달라진 자리고, 동시에 Prisma의 사용감을 가장 잘 보여 준다.

```
import { Injectable, NotFoundException } from "@nestjs/common";
import { PrismaService } from "../prisma/prisma.service";
import { CreateUserDto } from "../dto/create-user.dto";
import { User } from "@prisma/client"; // 타입이 schema.prisma에서 자동 생성됨

@Injectable()
export class UsersService {
  constructor(private readonly prisma: PrismaService) {}
```

- `import { User } from "@prisma/client"` — `User` 타입을 `@prisma/client` 에서 가져온다. 이 타입은 우리가 손으로 쓴 게 아니라 `schema.prisma` 의 `model User` 를 보고 `generate` 가 만든 것이다. 스키마를 고치면 이 타입도 다시 생성돼 자동으로 맞춰진다. 11장이 엔티티 클래스를 직접 타입으로 쓴 것과 대비되는 지점이다.
- `constructor(private readonly prisma: PrismaService)` — 생성자에서 `PrismaService` 하나만 주입받는다. 11장은 엔티티마다 `@InjectRepository(User)` 로 리포지토리를 따로 받았지만, Prisma는 이 객체 하나로 `prisma.user`, `prisma.post` 등 모든 모델에 접근한다.

```
create(dto: CreateUserDto): Promise<User> {
  return this.prisma.user.create({ data: dto });
}

findAll(): Promise<User[]> {
  return this.prisma.user.findMany({ include: { posts: true } });
}
```

- `this.prisma.user.create({ data: dto })` — 사용자를 한 명 만든다. `data` 에 넣을 값을 담는다. 반환값은 방금 만들어진 `User` 다.
- `this.prisma.user.findMany({ include: { posts: true } })` — 전체 사용자를 가져온다. `include: { posts: true }` 가 핵심이다. **연관된 글 목록까지 함께 로드**하라는 지시다. 이게 없으면 각 사용자에게 `posts` 배열이 빠진 채로 온다. 11장의 `find({ relations: ["posts"] })` 와 정확히 같은 역할을, `include` 라는 다른 단어로 한다.

`prisma.user.` 까지 치면 에디터가 `create` · `findMany` · `findUnique` 같은 메서드를 자동완성으로 띄운다.

`include` 안에 `posts` 를 칠 때도 마찬가지다. 이 자동완성이 전부 `schema.prisma` 에서 생성된 타입 덕분이다. 모델에 없는 필드를 적으면 컴파일 단계에서 걸린다.


```

async findOne(id: number): Promise<User> {
  const user = await this.prisma.user.findUnique({
    where: { id },
    include: { posts: true },
  });
  if (!user) throw new NotFoundException('사용자 ${id}를 찾을 수 없습니다.');
  return user;
}

```

- `findUnique({ where: { id } })` — 유일한 값(여기선 기본키 `id`)으로 한 건을 찾는다. 없으면 `null` 을 돌려 준다.
- `if (!user) throw new NotFoundException(.)` — 못 찾았으면 404를 던진다. NestJS가 이 예외를 받아 `{ statusCode: 404, message:... }` 형태의 응답으로 바꿔 내보낸다. 컨트롤러에서 상태코드를 손댈 필요가 없다.

```

async remove(id: number): Promise<void> {
  try {
    await this.prisma.user.delete({ where: { id } });
  } catch {
    throw new NotFoundException('사용자 ${id}를 찾을 수 없습니다.');
  }
}

```

- `delete({ where: { id } })` — 해당 사용자를 지운다. 그런데 없는 `id`를 지우려 하면 Prisma가 예외를 던진다. 그래서 `try/catch` 로 감싸 그 예외를 잡고, 404로 바꿔 던진다. 11장은 `delete` 의 결과에서 `result.affected` `=== 0` 을 검사했는데, Prisma는 예외 방식이라 처리 모양이 다르다. 결과는 같다 — 없는 사용자를 지우면 404다.

게시물 쪽 서비스는 이 코드와 판박이다. `prisma.user` 대신 `prisma.post` 를 쓰고, `include` 가 `{ author: true }` 로 바뀐 것만 다르다.

```

findAll(): Promise<Post[]> {
  return this.prisma.post.findMany({ include: { author: true } });
}

```

글 목록에 각 글의 작성자(`author`)를 객체로 끼워 넣는다. 사용자 조회가 글을 끌어왔다면, 글 조회는 작성자를 끌어온다. 1:N 관계의 양쪽을 그대로 뒤집은 모양이다.

컨트롤러와 DTO — 11장과 글자까지 같다

여기서 중요한 사실 하나. 컨트롤러와 DTO는 11장에서 한 글자도 바뀌지 않았다.

```

@Controller("users")
export class UsersController {
  constructor(private readonly userService: UsersService) {}

  @Post()
  create(@Body() dto: CreateUserDto) {
    return this.userService.create(dto);
  }

  @Get()
  findAll() {
    return this.userService.findAll();
  }

  @Get(":id")
  findOne(@Param("id") id: string) {
    return this.userService.findOne(+id);
  }

  @Delete(":id")
  remove(@Param("id") id: string) {
    return this.userService.remove(+id);
  }
}

```

컨트롤러는 "어떤 URL로 어떤 메서드가 들어오면 서비스의 어떤 함수를 부를지"만 정한다. 그 서비스 안에서 TypeORM을 쓰든 Prisma를 쓰든 컨트롤러는 알 바 아니다. `@Param("id") id: string`으로 받은 문자열을 `+id`로 숫자로 바꿔 넘기는 것까지 11장과 동일하다.

DTO도 마찬가지다. 검증 규칙은 ORM과 아무 상관이 없기 때문이다.

```

export class CreateUserDto {
  @IsEmail()
  email: string;

  @IsString()
  @MinLength(2)
  name: string;
}

```

`@IsEmail` · `@MinLength(2)` 같은 검증은 "들어온 요청 본문이 올바른가"를 보는 일이다. DB를 무엇으로 다루든 입력 검증은 그대로다. 이 검증을 켜는 건 부트스트랩의 전역 파이프다.

```

app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));

await app.listen(3002);
console.log("[Prisma] http://localhost:3002 (users, posts)");

```

- `whitelist: true` — DTO에 없는 필드는 잘라 낸다.
- `transform: true` — 들어온 값을 DTO 타입에 맞게 변환한다.
- `listen(3002)` — 포트 3002에서 요청을 받는다. 부트스트랩 맨 위 `import "dotenv/config"` 가 다른 `import` 보다 먼저 실행되며 `.env` 의 `DATABASE_URL` 을 `process.env` 에 올려 둔다. 그래야 Prisma가 연결할 때 그 값을 읽는다.

짚고 가기 — 컨트롤러·DTO·부트스트랩이 그대로라는 게 단순한 우연이 아니다. NestJS가 계층을 나눠 둔 덕분이 다. "들어오는 요청을 받고(컨트롤러), 검증하고(DTO), 처리한다(서비스)"에서 DB는 맨 안쪽 서비스의 관심사다. 그래서 ORM을 바꿔도 바깥 계층은 흔들리지 않는다. 이게 11장과 12장을 나란히 만든 진짜 이유다.

동작 확인 (curl)

서버를 띄운 뒤(`npm run start:dev`) 다른 터미널에서 요청을 보낸다. 사용자를 먼저 만들어야 글에 붙일 `authorId` 가 생긴다.

```
curl -X POST http://localhost:3002/users \
  -H "Content-Type: application/json" \
  -d '{"email":"alice@example.com","name":"앨리스"}'

curl -X POST http://localhost:3002/posts \
  -H "Content-Type: application/json" \
  -d '{"title":"첫 글","content":"안녕하세요","authorId":1}'

curl http://localhost:3002/users          # 각 user에 posts 배열 포함 (include)
curl http://localhost:3002/posts         # 각 post에 author 객체 포함 (include)
curl http://localhost:3002/users/1
curl -X DELETE http://localhost:3002/posts/1
```

`GET /users` 의 응답에서 각 사용자에게 `posts` 배열이 붙어 있으면 `include` 가 동작했다는 뜻이다. `GET /posts` 에서 는 각 글에 `author` 객체가 붙는다. `npm run prisma:studio` 로 GUI를 열면 같은 데이터를 표 형태로도 확인할 수 있다.

TypeORM(11) ↔ Prisma(12) 대응표

관심사	TypeORM (11)	Prisma (12)
스키마 정의	<code>@Entity</code> 데코레이터 클래스	<code>schema.prisma</code> 의 <code>model</code>
타입	엔티티 클래스를 그대로 사용	<code>prisma generate</code> 가 타입 생성
접속 정보	<code>forRoot({ host, port,... })</code>	<code>.env</code> 의 <code>DATABASE_URL</code> 한 줄
DB 접근	<code>@InjectRepository</code> → <code>Repository<T></code>	<code>PrismaService</code> 1개 → <code>prisma.user.*</code>
모듈 등록	모듈마다 <code>forFeature([.])</code>	<code>@Global</code> <code>PrismaModule</code> 한 번
관계 로드	<code>find({ relations: ["posts"] })</code>	<code>findMany({ include: { posts: true } })</code>
없는 건 삭제	<code>result.affected === 0</code> 검사	<code>try/catch</code> 로 예외 처리
테이블 생성	<code>synchronize: true</code> (자동)	<code>prisma migrate dev</code> (마이그레이션 기록)
DB GUI	외부 툴	<code>prisma studio</code> 내장

흔한 실수

- `generate` 를 안 하고 실행한다. → `@prisma/client` 의 타입이 비어 있어 `User · Post` 를 못 찾고 빌드가 실패한다. 설치 직후엔 `npm run prisma:migrate` (또는 `prisma:generate`) 먼저.
- `.env` 의 `DATABASE_URL` 을 안 고친다. → `USER:PASSWORD` 자리가 그대로면 인증 실패로 연결이 안 된다. 비밀번호 없는 계정은 `:PASSWORD` 를 통째로 뺀다.
- `createdb` 를 빼먹는다. → `nest_prisma` DB가 없으면 마이그레이션이 "database does not exist"로 멈춘다. `createdb nest_prisma` 부터.
- `include` 를 빼고 관계 데이터를 기대한다. → `findMany()` 만 부르면 `posts · author` 가 응답에 없다. 연관 데이터는 명시해야 따라온다.
- 스키마를 고친 뒤 `migrate`를 안 돌린다. → 코드의 타입과 실제 DB 테이블이 어긋난다. `schema.prisma` 를 손냈으면 `prisma migrate dev` 로 둘을 다시 맞춘다.

직접 해보기

1. `schema.prisma` 의 `User` 에 `bio String?` 필드를 추가하자(? 는 선택 컬럼). 그다음 `npx prisma migrate dev --name add-bio` 를 돌려 마이그레이션이 새로 생기는지, `User` 타입에 `bio` 가 붙는지 확인한다.

2. 게시물 서비스에 `findByAuthor(authorId: number)` 를 추가하자. `this.prisma.post.findMany({ where: { authorId } })` 로 특정 작성자의 글만 거른다. 컨트롤러에 `GET /posts?authorId=1` 라우트도 함께 붙여 본다.
3. `npm run prisma:studio` 로 Studio를 열어 글을 하나 직접 추가한 뒤, `curl http://localhost:3002/posts` 로 그 글이 API에도 보이는지 확인하자. 스키마 한 곳이 DB·코드·GUI를 모두 묶고 있음을 눈으로 확인하는 실험이다.

정리

- Prisma는 `schema.prisma` 파일 하나를 진실의 원천으로 삼고, 거기서 마이그레이션과 타입을 생성한다. 엔티티 클래스가 곧 스키마였던 TypeORM과 갈리는 출발점이다.
- 워크플로우는 2단계다 — `prisma migrate dev` 로 DB에 테이블을 만들고, `prisma generate` 로 타입 클라이언트를 만든다. 스키마를 고치면 둘을 다시 돌려 맞춘다.
- `PrismaService` 는 `PrismaClient` 를 상속해 NestJS 생명주기에 연결하고, `@Global` `PrismaModule` 이 그걸 전역에 푼다. 모듈마다 `forFeature` 를 적던 TypeORM보다 등록이 간결하다.
- ORM을 바꿔도 컨트롤러·DTO·부트스트랩은 그대로다. 계층이 나뉘어 있어 DB 교체가 서비스 안쪽에 갇히기 때문이다.

다음 장(13·인증)에서는 이 게시판에 회원가입과 로그인을 붙인다. 비밀번호를 bcrypt로 해싱하고, JWT로 로그인 상태를 증명하며, Guard로 보호된 라우트를 만든다.